

Tutorium 2

Grundlagen Haskell

Grundlagen der Programmierung

15. Juni 2026

Theorie: Funktionen & Typen

Praxis

Funktionen in Haskell

Funktionen in Haskell verhalten sich ähnlich wie mathematische Funktionen. Sie bestehen aus einem **Bezeichner**, **Parametern** und einem **Ausdruck**.

Beispiel: Addition

```
1  -- Definition ohne explizite Typen
2  add x y = x + y
```

- ▶ **Bezeichner:** `add`
- ▶ **Parameter:** `x` und `y`
- ▶ **Ausdruck:** `x + y`

In Haskell kann (und sollte) man optional eine **Typsignatur** angeben.

Funktion mit Typsignatur

```
1  -- func' :: Parameter 1 -> Parameter 2 -> Parameter 3 ->
   Rueckgabetyt
2  func' :: Integer -> Integer -> String -> Integer
3  func' x y z = x + y
```

- ▶ `::` bedeutet “hat den Typ”.
- ▶ Die Typen der Parameter werden nacheinander mit `->` getrennt.
- ▶ Der **letzte Typ** in der Kette ist immer der **Rückgabetyt**.

Ein rekursiver Algorithmus benötigt immer einen **Basisfall**, damit er terminieren kann. Ansonsten entsteht eine Endlosschleife!

Beispiel: Fakultät

```
1  -- 5! = 5 * 4! = 5 * (4 * 3!) ...
2  fak :: Integer -> Integer
3  fak 0 = 1                      -- Basisfall
4  fak x = x * fak (x-1)          -- Rekursiver Aufruf
```

Live Demo



haskell.hs

Wir programmieren Pattern Matching und verschiedene Rekursionsarten zusammen im Code!

Aufgaben:

1. Wir wollen die Zahlen 1 - 3 als Wort ausgeben ("eins", "zwei"), sonst einen Fehler.
2. Fakultät mit Akkumulator (Endrekursion) definieren.
3. Welche Rekursionsarten liegen vor?